Universität Rostock



SOFTWAREENTWURF

Joern Ploennigs · AI4SC

Midjourney: Waterfall in chinese mountain range

Hörsaalfrage

Was entsteht zuerst — Code oder Plan?



Die Lücke: Anforderung → Code

Auftraggeber sagt:

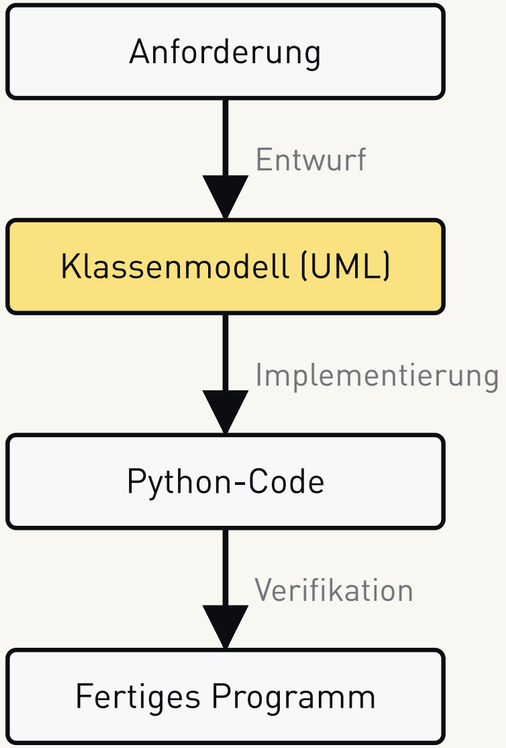
> „Ich brauche ein Programm, das den Wasserverbrauch eines Gebäudes pro Raum berechnet."

Entwickler braucht:

- Welche *Klassen* gibt es?
- Welche *Attribute* (mit Typen)?
- Welche *Methoden* (Signaturen)?
- Wie hängen die *Teile zusammen*?

Entwurf schliesst die Lücke

Der Entwurf übersetzt eine Anforderung in eine strukturierte Programmbeschreibung *bevor* Code geschrieben wird.



Softwareentwurf als Projektplan

- Software wird selten allein entwickelt, sondern oft im Team über einen längeren Zeitraum.
- Dafür wird ein Projektplan benötigt, der beschreibt, wie die Software arbeitsteilig entwickelt werden soll.
- Das Erstellen dieses Projektplans nennt man *Softwareentwurf*.
- Der Ablauf ähnelt stark dem *Entwurfsvorgehen im Umwelt- und Bauingenieurwesen*.
- Der Projektplan umfasst:
  - den *Klassenentwurf* (Bauplan)
  - die *Programmiervorgehen* (Bauablaufplanung)

Softwareentwurf — Phasen

- *Anforderungsdefinition* – Definition welche Funktionen und Randbedingungen realisiert werden
- *Entwurf*
  - Klassenentwurf der Software
  - Ausführungsplanung – Was wird wann, durch wen, und wie implementiert
- *Ausführung*
  - Implementation – Programmieren der einzelnen Klassen & Module
  - Integration – Zusammenführen der Module zur fertigen Lösung
- *Abnahme* – Test des fertigen Softwareprogramms
  - Modultest – Unit-Test von Modulen
  - Integrationstest – Kombination von Modulen testen
  - Systemtest – Gesamtheit der Module testen

Vergleich: Software- vs. Bauingenieurwesen

Softwareentwurf

- *Anforderungsdefinition:* Definition welche Funktionen und Randbedingungen realisiert werden
- *Entwurf*
  - Modulentwurf – Definition welche Teile die Software hat
  - Klassenentwurf – Definition in welche Klassen die Software strukturiert ist
  - Ausführungsplanung – Was wird wann, durch wen, und wie implementiert

Bauingenieurwesen

- *Ausschreibung:* Dokumentieren welche Funktionen und Randbedingungen das Bauwerk erfüllen
- *Entwurf*
  - Architektur und Grobentwurf – Definition wie das Bauwerk grob strukturiert ist
  - Fein- und Gewerkplanung – Definition wie Teile und die Gewerke umgesetzt werden
  - Ausführungsplanung – Was wird wann, durch wen, und wie gebaut

Vergleich: Software- vs. Bauingenieurwesen (Fortsetzung)

Softwareentwurf

- *Ausführung*
  - Implementation – Programmieren der einzelnen Klassen & Module
  - Integration – Zusammenführen der Module zur fertigen Lösung
- *Abnahme*
  - Modultest – Unit-Test von Modulen
  - Integrationstest – Kombination von Modulen testen
  - Systemtest – Gesamtheit der Module testen

Bauingenieurwesen

- *Ausführung*
  - Einzelne Gewerke werden umgesetzt
  - Gewerke im Bau integrieren, wie Energie und Wasser mit Klima integrieren
- *Abnahme*
  - Abnahme einzelner Gewerke
  - Test mehrerer Gewerke gemeinsam
  - Gesamtbauwerk abnehmen

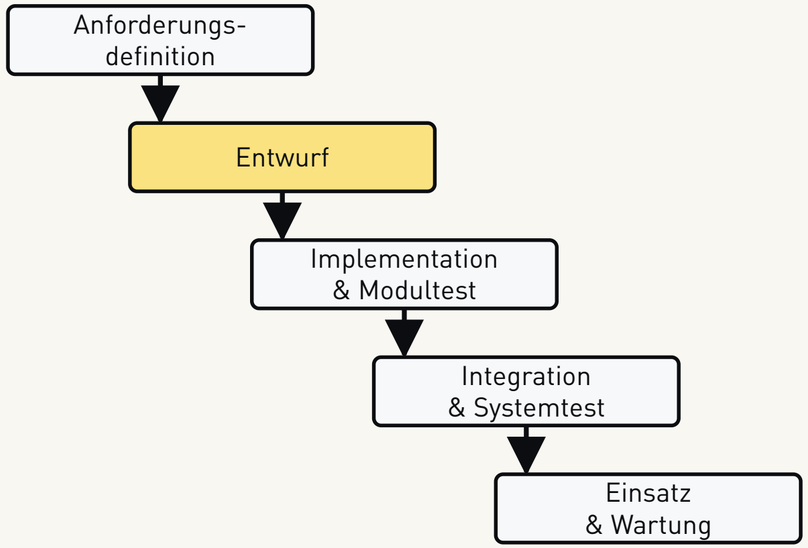
Welches Vorgehensmodell passt?

Vier Modelle, verglichen nach drei Kriterien: Wann passt es, wann wird geprüft, welches Risiko bleibt?

Modell	Passt bei	Geprüft	Risiko
Wasserfall	stabilen Anforderungen, Ausschreibung großer Systeme	nach jeder Phase, Abnahme am Ende	Fehler früher Phasen fallen spät auf
V-Modell	sicherheitskritischer Software (Auto, Flugzeug)	Tests schon im Entwicklungsarm definiert	aufwendiger als andere Modelle
Agil	sich ständig ändernden Anforderungen	pro Iteration, mit Feedback zur lauffähigen Version	Entwurfsfehler können Neuentwurf erzwingen
Agentisch	KI-Coding-Assistent verfügbar	Mensch verifiziert nach jeder Generierung	Verstehen und Prüfen liegen beim Ingenieur

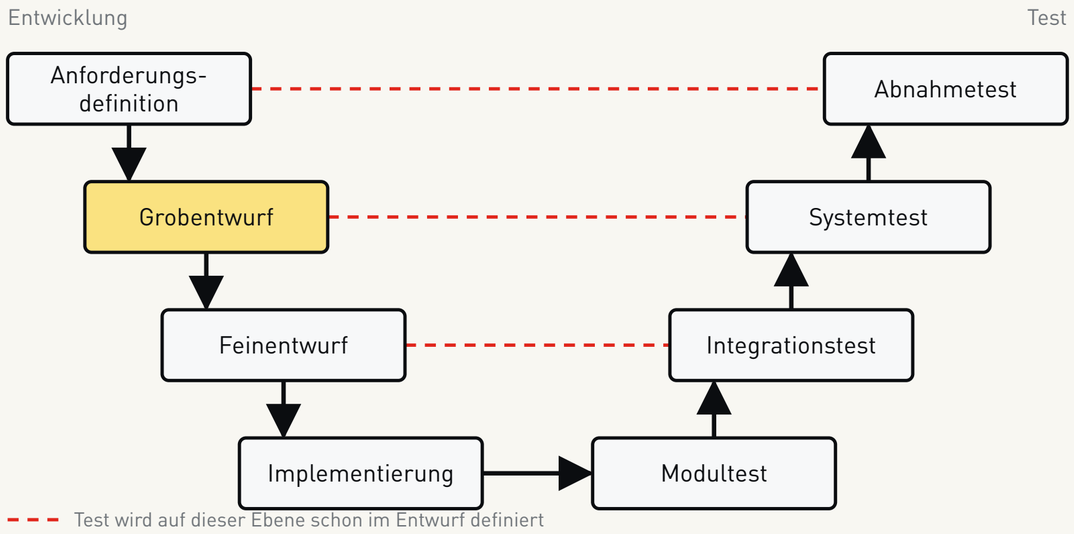
Wasserfallmethode

- Traditionelles Modell das häufig noch in der Ausschreibung großer Systeme gefordert wird
- Entwicklung ist in mehrere sequenzielle Schritte aufgeteilt und jeder Schritt muss vor dem nächsten beendet werden
- Benutzerbeteiligung nur in der Anforderungsdefinition
- Jeder Aktivität wird dokumentiert → Gut geeignet für Ausschreibungen (nach der Anforderungsdefinition oder dem Entwurf, ISO 9000)



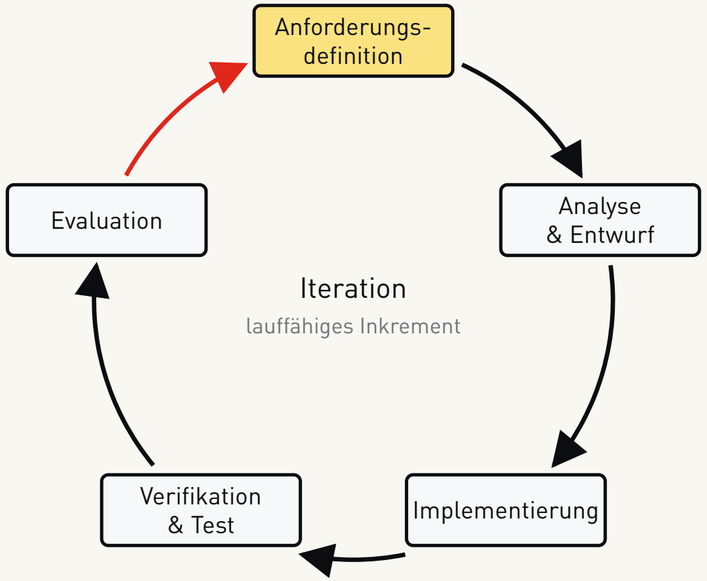
V-Methode

- Primär verwendet bei der Entwicklung sicherheitskritischer Software (Auto, Flugzeug, etc.)
- Separiert Entwicklungs- (linke Seite) und Testaktivitäten (rechte Seite)
- Tests werden schon im Entwicklungsarm definiert
- Ziel einer hohen Testabdeckung
- Benutzerbeteiligung in der Anforderungsdefinition und im Abnahmetest



Agile Methode

- Moderne Methode um mit sich an ständig ändernden Anforderungen anpassen
- Ziel der schrittweisen Entwicklung der Lösung, um Aufwand und Komplexität einzelner Schritte in Grenzen zu halten
- Startet mit einer einfachen und ausbaufähigen Implementierung (MVP – Minimal Viable Produkt)
- Schrittweise Erweiterung und Verbesserung des Produktes mit regelmäßigen Releases (meist alle 3 Monate)
- Entwurfsfehler der ersten Iterationen können zu kompletten Neuentwurf führen

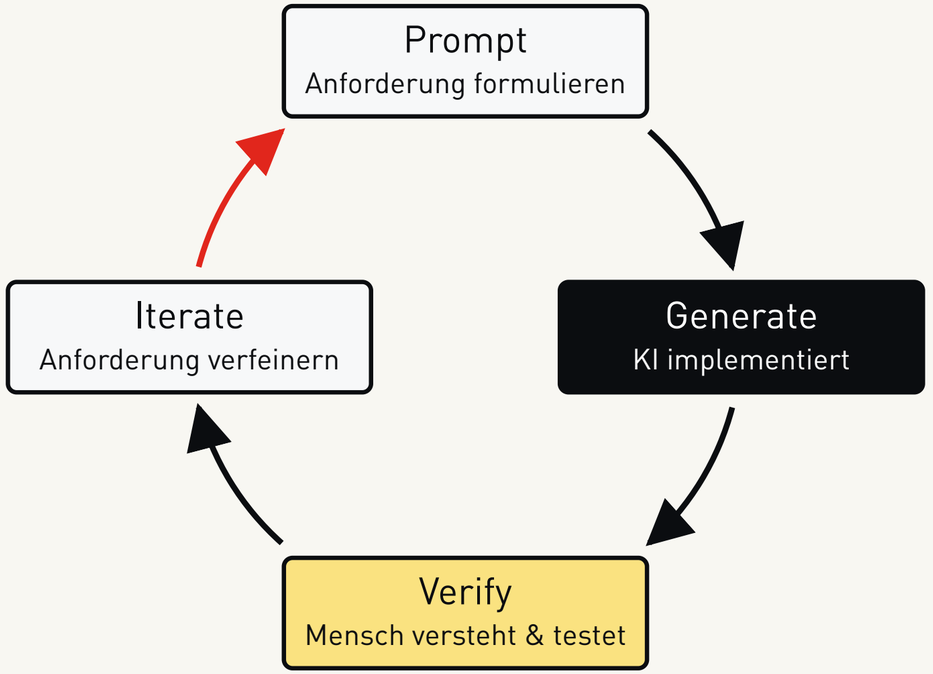


Agentischer Entwicklungs-Loop

KI übernimmt die Implementierungsphase — der Ingenieur steuert und verifiziert:

- *Prompt* = Anforderungsspezifikation
- *Generate* = Implementierungssprint (KI)
- *Verify* = Verstehen & Testen (Mensch)
- *Iterate* = Prompt verfeinern

Kernkompetenz: Verstehen und Verifizieren



Vier Vorgehensmodelle im Vergleich

Wasserfall

- Sequenziell
- Dokumentiert
- Stabile Anf.

V-Modell

- Testbasiert
- Sicherheit
- Hohe Qualität

Agil

- Iterativ
- Flexibel
- Schnell

Agentisch

- KI generiert
- Prompt = Spec
- Verify = Kern

Modellierung mit Objekten

- Unsere Wahrnehmung und unser Denken besteht darauf Objekte zu erkennen. Das setzt voraus, dass wir das Objekt von anderen Objekten trennen können (z.B. Stuhl und Tisch)
- Das Nachbilden dieser Wahrnehmung und dieses Weltverständnisses im Computer, nennt man *Modellierung*.
- Da unsere Wahrnehmung auf Objekten basiert, ist es naheliegend für die Modellierung auch Objekte zu nutzen.
- Die Objekte in der Welt interagieren, dieses Verhalten müssen wir auch modellieren.

> **MODELL**
>
> Ein Modell ist eine abstrakte, vereinfachte Abbildung eines Systems (meist aus der Realität).

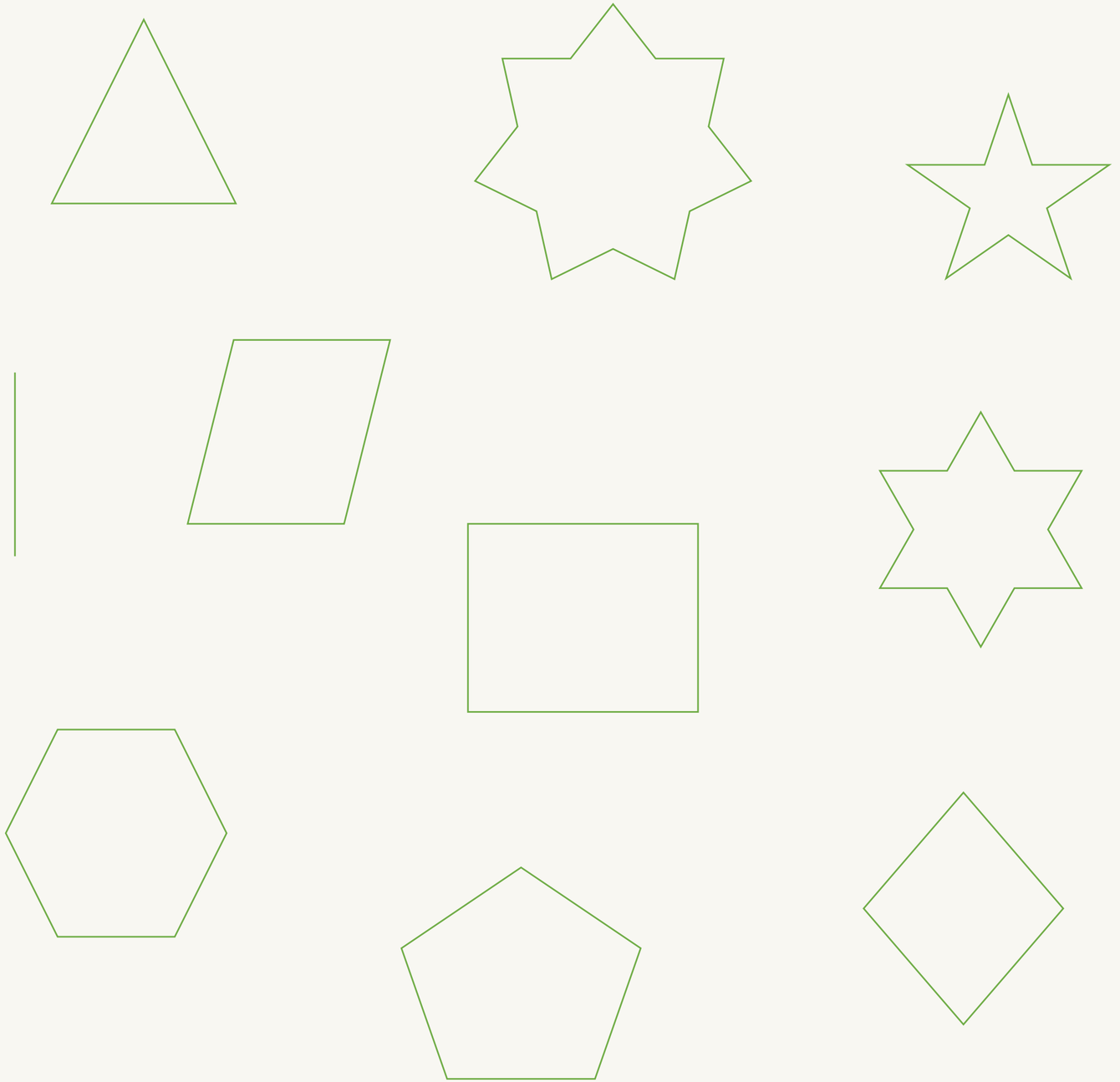
Hörsaalfrage

Was haben diese Elemente gemein?



Hörsaalfrage

- Das sind alles *geometrische Objekte*
- Sie bestehen alle aus *Punkten*, die mit *Linien* verbunden sind
- Das können wir bei der Modellierung ausnutzen und Klassen so definieren, dass sie die *Gemeinsamkeit* ausnutzen



Objektorientierten Softwareentwurf

Im objektorientierten Softwareentwurf wird ein Programm so entworfen, dass es nur aus *Objekten* besteht.

- Der Softwareentwurf gleicht dabei stark dem Vorgehen von Ingenieuren beim Entwurf von Plänen
- Die übliche Modellierungssprache sind *UML Diagramme*.
- Man modelliert im Entwurf:
  - wie Objekte in Form von *Klassen* definiert sind,
  - welche *Attribute und Methoden* sie besitzen,
  - wie diese Klassen aufeinander aufbauen (*Vererbung*),
  - wie sie miteinander in statischer Beziehung stehen (*Referenzen*)
  - und wie sie dynamisch interagieren (*Verhalten*)

UML — Entwurfssprache für Software

- *UML (Unified Modelling Language)* ist ein ISO-Standard zur Beschreibung von Softwareentwürfen
- Er wird genutzt für den Entwurf, Ausschreibung, Implementation und Dokumentation von:
  - Funktionalitäten
  - Strukturen
  - Businessprozessen
  - Nutzerinteraktionen

Der Standard umfasst 13 Diagrammtypen, die typischsten:

- *Klassendiagramme* (Für die Klassen- und Datenstruktur)
- *Programmablaufplan* (Für allgemeine Abläufe und Algorithmen)
- *Sequenzdiagramme* (Für Interaktionen)
- *Use-Case Diagramme* (Für Nutzer und Nutzfälle)

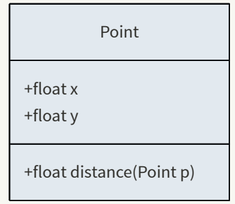
UML Klasse — Aufbau

Eine Klasse besteht aus drei Ebenen:

01 *Klassenname* (oben)

02 *Attribute* mit Datentyp (mitte)

03 *Methoden* mit Signatur (unten)



Attribut- und Methoden-Notation

Attribute:

```
attributname: Typ
```

Beispiele:

```
name: str
area: float
rooms: list[Room]
```

Methoden:

```
method(param: Typ) Rückgabetyp
```

Beispiele:

```
calculate_area() float
get_rooms() list[Room]
add_room(r: Room)
```

UML Klassendiagramm

- Das am *häufigsten verwendete Diagramm* im objektorientierten Modellieren
- Im Klassendiagramm modelliert werden:
  - Klassen mit: Attributen, Methoden
  - Beziehungen
  - Hierarchien & Vererbungen

Referenzen

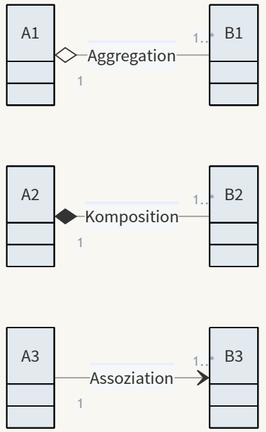
- In Programmen stehen Objekte meist im Zusammenhang. z.B. besteht eine Linie aus zwei Punkten.
- Diesen Zusammenhang zweier Objekt-Klassen bezeichnet man als ⇆ *Referenz*.
- Referenzen in Python erstellt man als Attribut vom Datentyp des anderen Objektes.

```python
class Line:
    def __init__(self, start, end):
        self.start = start
        self.end = end

    def length(self):
        return self.start.distance(self.end)
```

Klassendiagramm - Referenzen

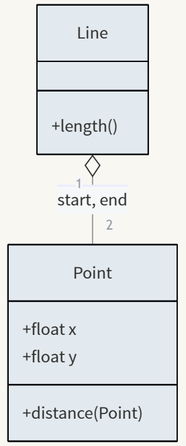
- *Referenzen* werden in UML mit Linien zwischen Objekten gekennzeichnet. Die Art der Linie und des Pfeils geben den Typ der Referenz an.
- Man unterscheidet in UML die Referenzen in der Art der *Besitzverhältnisse*:
  - *Aggregation* - ein Objekt ist Teil eines anderen und kann ohne ihn existieren
  - *Komposition* - kann nicht ohne diesen existieren
  - *Assoziation* - komplett unabhängig
- *Zahlen* an den Referenzen geben die *Multiplizität* an, also wie viele Objekte in dieser Relation im Zusammenhang stehen.



Multiplizität

Zahlen an Pfeilen geben die *Multiplizität* an — wie viele Objekte in Relation stehen:

Notation	Bedeutung
`1`	Genau eins
`0..*`	Null oder mehr
`1..*`	Ein oder mehr
`0..1`	Keins oder eins

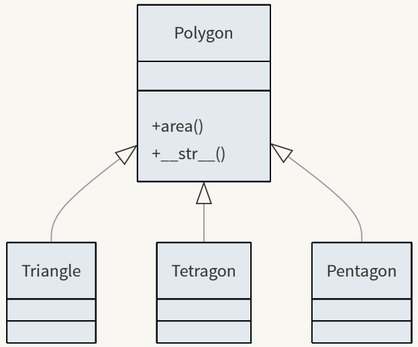


Vererbung

- Gleiche Objekte teilen oft auch ähnliche Eigenschaften und Methoden. z.B. bestehen alle geometrischen Elemente aus Punkten die mit Linien verbunden sind
- Um diese Attribute und Methoden nicht jedes mal neu definieren zu müssen dabei ggf. Fehler zu machen, gibt es die *Vererbung* in der OOP
- OOP erlaubt das Definieren von spezialisierten *Unterklassen*, bzw. von generalisierten *Oberklassen*.
- Die Unterklasse (Spezialklasse) besitzt die Methoden und Attribute der generalisierten Oberklasse (Basisklasse).
- Angelehnt an die Genetik auch *„Vererbung"* genannt

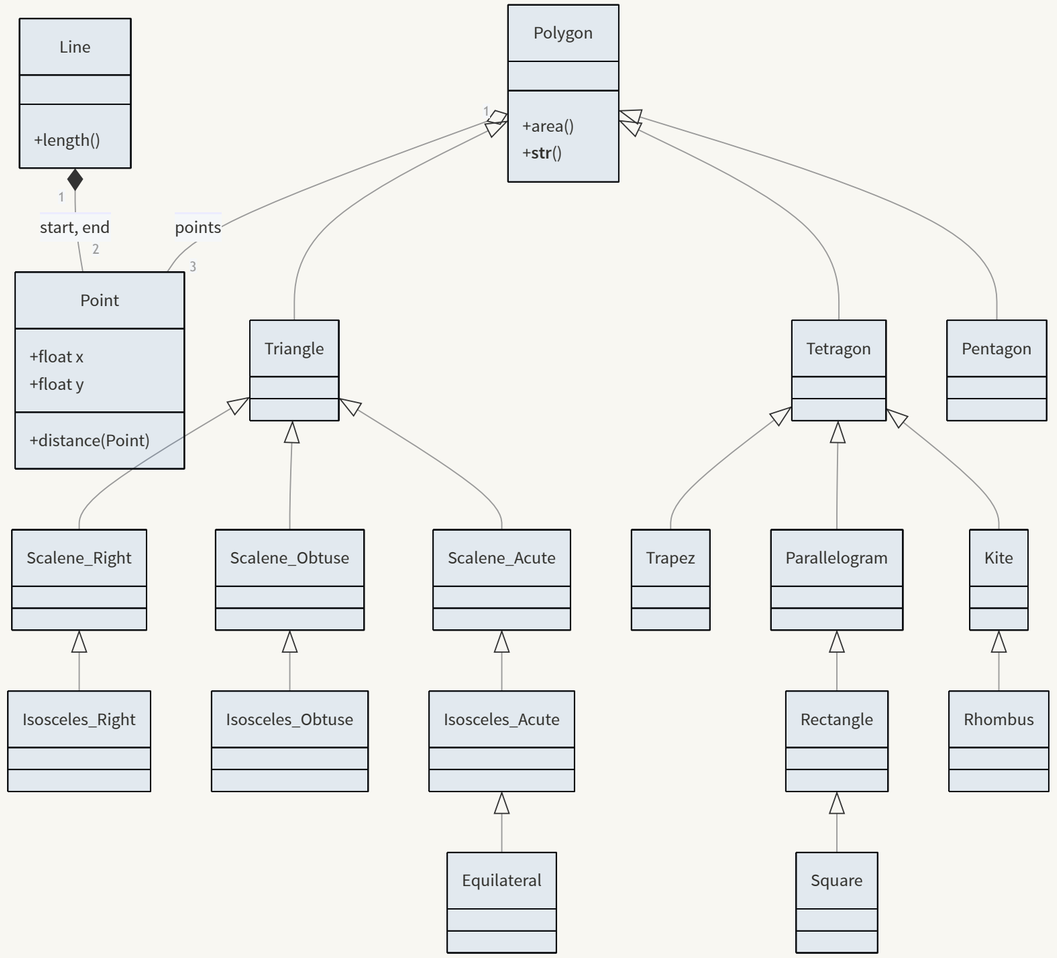
Klassendiagramm – Vererbung

- *Vererbung* wird in UML durch eine Referenz mit einem *gefüllten Dreieck* bei der Oberklasse gezeichnet.
- Um zum Beispiel auszudrücken, dass `Triangle`, `Tetragon` und `Pentagon` Unterklassen des Polygons sind, können wir zeichnen.



Klassendiagramm - Gesamtbeispiel

- *Klassendiagramme* können recht schnell groß werden
- Sie eignen sich sehr gut um das *(Daten-) Modell* darzustellen



BIM Beispiel — Schritt 1: Anforderung formulieren

Auftraggeber formuliert:

> „Ein Gebäude hat mehrere Räume. Jeder Raum hat einen Namen, eine Fläche und eine Liste von Fenstern. Ein Fenster hat eine Breite und Höhe. Das Programm soll die Gesamtfensterfläche eines Raumes berechnen."

Ingenieursfragen für den Entwurf:

- Welche *Klassen* brauchen wir?
- Welche *Attribute* mit welchen Typen?
- Welche *Methoden* mit Signaturen?
- Welche *Beziehungen* zwischen Klassen?

→ Diese Fragen beantwortet das UML-Diagramm

BIM Beispiel — Schritt 2: Anforderung → UML

```
classDiagram
    class Building {
        name: str
        address: str
        get_rooms() list[Room]
    }
    class Room {
        name: str
        area: float
        get_window_area() float
    }
    class Window {
        width: float
        height: float
```

BIM Beispiel — Schritt 3: UML → KI-Prompt → Verifizieren

KI-Prompt (aus UML abgeleitet):

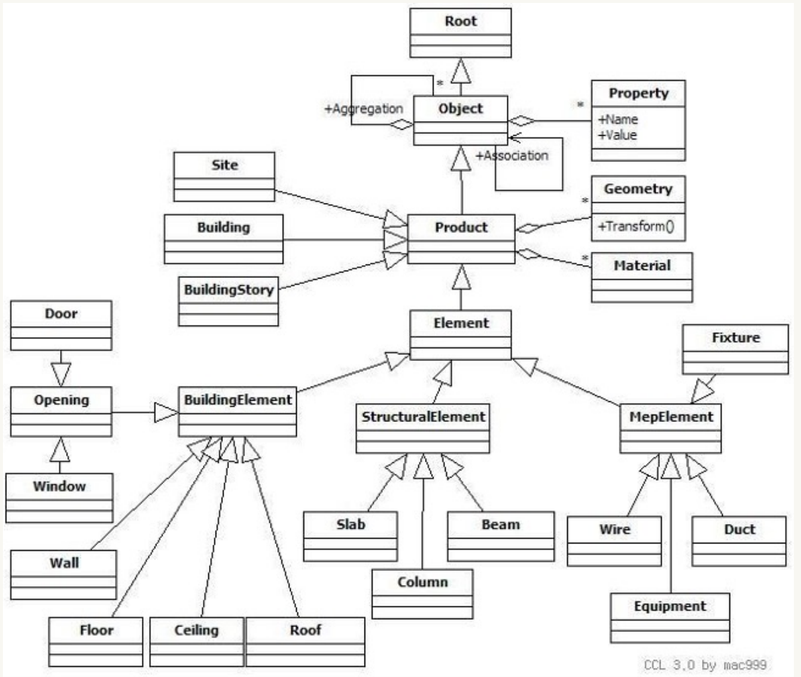
> Erstelle Python-Klassen für `Building`, `Room` und `Window`. Building enthält eine Liste von `Room`-Objekten. `Room` hat `name: str`, `area: float` und eine Liste von Fenstern. `Window` hat `width: float`, `height: float` und `get_area() -> float`. `Room` hat `get_window_area() -> float`.

Verifikation gegen UML:

- ✓ Alle drei Klassen vorhanden?
- ✓ Attribute mit korrekten Typen?
- ✓ Methoden-Signaturen korrekt?
- ✓ Beziehungen implementiert?
- ✓ Multiplizität berücksichtigt?

Anwendungsbeispiel — BIM (Building Information Models)

- Bauzeichnungen werden heutzutage als *IFC (Industry Foundation Classes)* gespeichert
- *IFC* ist ein objektorientiertes Modell mit Vererbung, Spezialisierung und Generalisierung
- Es wird u.a. in *UML* dokumentiert



Polygon Klassen

QUESTIONS